

GalaxEye — Backend Engineer, ML Systems

Design Note & Problem-Solving Answers

GalaxEye Take-Home — Sangeeta Upadhye

PART 1

Architecture

I split the system into four components:

1. Ingestion API — receives the tile
2. Model service — runs the CNN classifier
3. Storage layer — SQLite, saves every result
4. Query layer — lets analyst search results (*designed, not built — stubbed per assignment instructions*)

How a tile flows through the system:

- Tile image hits a single API endpoint
- API resizes it to 64×64 (what the model expects)
- Image passed into the CNN → outputs a probability for each of the 7 classes
- Top class = the prediction, but **full probability list is kept**, not discarded
- One row written to SQLite: filename, predicted class, confidence, full probability breakdown, model version, timestamp

Query layer (not built, but designed):

- Would be a read-only endpoint — filter by class, confidence, or date
- Reads from the same SQLite table already being written to
- Schema already supports this — adding it later needs zero changes to existing code

Why single-process, not split into microservices:

- Runs on isolated hardware, no ops team watching it → fewer moving parts = fewer things to break unsupervised
- Can split into separate services later *if* tile volume ever demands it — didn't want to over-engineer for scale with no evidence it's needed

How "offline" shaped the design — 2 direct decisions:

- Model weights are baked into the deployment itself, not downloaded at startup → zero internet dependency, ever
- Chose SQLite over a real DB server → can't assume isolated hardware has a database server installed/permitted/maintained; a single file any tool can open later is the safer default

Trade-offs

1. Handling low-confidence predictions

- Options considered:
 - Accept everything blindly
 - Hard-reject anything below a confidence threshold
 - Store everything, but make confidence filterable (*chosen*)
- Why: rejecting outright throws away data the analyst might still want
- Real evidence backing this: an earlier broken version of my pipeline was 99% confident while being wrong 86% of the time — proof that confidence scores alone don't mean the model is trustworthy
- Conclusion: surface the number, let a human decide, don't bake in a judgment call for them

2. What to store

- Options considered:
 - Just the winning label
 - Full probability distribution + metadata (*chosen*)
- Why: extra storage cost is negligible (a few hundred bytes/row), but it's the difference between being able to ask "was Forest a close second guess?" later — versus not being able to answer that at all

3. What "querying" should mean

- Options considered:
 - Hand analyst raw SQLite file access (e.g. via free tool "DB Browser for SQLite")
 - Build a proper REST query endpoint
- Chosen approach: start with raw file access — zero code, works immediately, doesn't assume the analyst can install new software on locked-down hardware
- Build the REST endpoint later only if analysts actually want a nicer interface — ship the cheap version first, see if it's enough

Assumptions & Open Questions

Assumptions made:

- Hardware is CPU-only, no GPU at the deployment site
- Tiles arrive one at a time, not in large real-time bursts
- "Offline" = no internet at runtime, but a one-time connected setup (installing the software itself) is acceptable
- Small number of analysts, not many concurrent users — part of why SQLite is good enough

Questions I'd ask GalaxEye:

1. How isolated is "isolated" — permanently air-gapped, or occasionally connected for updates?
2. Roughly how many tiles per day, realistically?
3. Who is "the analyst" in practice — SQL-comfortable, or needs a polished UI?
4. Are these 7 land-use categories the real ones that matter, or just a stand-in for this exercise?
5. Does a wrong classification ever carry real operational/safety consequences? — would change how aggressively I'd flag low-confidence predictions vs. just logging them

PART 2

(working code + README):

<https://github.com/sangeetaupadhye/galaxeye-tile-service>

PART 3

PART 2 — see repo linked above (code, trained model, README with setup/run instructions and what's stubbed)

Q1: Wrong 30% of the time — what now?

First I'd check where the errors are coming from — is it spread evenly across all classes, or stuck on 1-2 classes like we saw with Highway/River? That tells me if it's a real weak spot or just random noise.

Whether 70% is "good enough" depends on what it's used for. If a human still checks the result before acting on it, 70% is fine — it's still saving time. If it's fully automatic with no check, 70% is risky. I'd use confidence score as a filter — trust the high-confidence ones, send low-confidence ones to a human.

Q2: No one's watching for a month — how do I know it still works?

Since I already store confidence scores for every prediction, I'd track the average confidence over time. If it suddenly drops, something changed.

I'd also keep a few tiles with known correct answers, and run them through the model once in a while as a check — if it starts getting those wrong, something broke.

Basic stuff too — check the API isn't crashing, disk isn't full, DB is still writing rows.

Q3: Results look wrong — how do I debug it?

1. Look at the actual tile image myself first — maybe it's not even wrong, just a hard case
2. Check the stored row — do the probabilities add up, is the right filename attached
3. Send that same tile through the API again manually, see it happen live
4. Check if the image is being resized/processed the same way as during training — small mismatches here cause silent wrong answers
5. Only after all that, consider it might just be a real model limitation

Q4: Weakest part of the design?

Highway and River get confused with each other — they look similar at 64x64 resolution. That's a real model limit, not a bug.

On the engineering side, SQLite only allows one writer at a time, so that'd be the first thing to break if there were multiple tile sources writing at once.

But honestly, the thing most likely to break first isn't the model or the database — it's that there's no way to actually browse the results yet. They're just piling up in a database file with no way to view them without opening it manually.